AI-Powered Intelligent Job Search and Career System

Data Science and AI Project

Milestone 1 Report

Group 4

Prepared By:

Gaurav Kumar (22f1001105)
Dev Gupta (22f2000888)
Abhinav Ohri (24f1002064)
Pranav N (22f2000117)
Praveena N (22f3001454)

Index

1. Problem Statement, Stakeholders and Scope

The problem addressed in this project is the fragmented and manual nature of the job search process. Candidates often have to upload or rewrite resumes, search across multiple job portals, compare job descriptions, and guess which roles fit their background. These steps are handled through separate tools, so the candidate has to manually connect the whole process.

The proposed system, **AI-Powered Intelligent Job Search and Career Support System**, aims to assist this process by building a web application. The core system will parse an uploaded resume, create a structured candidate profile, recommend suitable career roles, and rank relevant job opportunities based on profile fit. If time permits, additional modules such as ATS-style scoring, resume tailoring, and cover letter generation will be added as good-to-have features.

The primary stakeholders are active job seekers, fresh graduates, career changers, and working professionals who want better guidance while applying to suitable roles. The secondary stakeholders are recruiters and employers, who may indirectly benefit from receiving more relevant and better-structured applications.

The project is in scope for the following:

Area	Included in Scope	Requirement
Resume Ingestion	Parsing uploaded resumes and extracting structured fields such as skills, education, experience, projects, certifications, and job titles	Mandatory
Candidate Profile Creation	Building a persistent structured profile that later modules can use	Mandatory
Career Recommendation	Recommending suitable roles and career directions based on the candidate's profile	Mandatory
Job Discovery and Ranking	Collecting job descriptions from public datasets, saved postings, and allowed public pages, then ranking them by candidate-profile fit.	Mandatory
Resume Tailoring	Modifying resume content for a specific job description while preserving factual correctness. The LLM will be restricted to rephrasing, reordering, and emphasizing content already present in the candidate's structured profile.	Good to have
ATS Scoring	Providing an ATS-style compatibility score (0–100) that will be computed as a weighted combination of skills, keywords, role fit, and formatting.	Good to have
Cover Letter Generation	Creating role-specific cover letters grounded in the candidate profile and job description	Good to have
Web interface	A usable interface where candidates can upload resumes, view structured profiles, see career recommendations, and view ranked job matches.	Mandatory

The project is **explicitly out of scope** for the following:

- Checking the authenticity or genuineness of posted jobs, including background verification of employers or companies.
- Fully autonomous auto-apply without user review.
- Extracting personal contact information of recruiters
- Direct messaging to recruiters for users.
- Interview preparation
- Fabricating experience or credentials

2. Why the Problem Matters and Where It Arises

This problem matters because job search has become increasingly competitive, AI-mediated, and time-consuming. LinkedIn's 2026 research reports that more than half of people globally are looking for a new role and 65% of people say finding a job has become more challenging. The same report also states that 93% of recruiters plan to increase their use of AI in 2026, showing that both sides of the hiring process are becoming more AI-driven.

This problem arises in online recruitment settings where job seekers use platforms such as LinkedIn, Naukri, Indeed, Glassdoor, and company career pages. However, the existence of many portals does not automatically make the process easier. Candidates still face three practical difficulties:

- **Discovery overload:** There are many job listings, but candidates may not know which ones truly match their profile.
- **Random applications:** Due to high competition, candidates often apply to many jobs without knowing which roles are actually suitable for them.
- **Lack of feedback:** Candidates rarely get clear feedback on whether their profile is a good fit for a specific role.

Therefore, this project is important because it helps reduce the gap between job availability and candidate readiness. The aim is not to fully automate the job search, but to support candidates in identifying better-fit roles and preparing stronger application material where possible.

3. How the Problem Is Currently Addressed

At present, the job search workflow is supported by both commercial tools and academic research. Commercial tools usually support only one part of the process, while academic research focuses on specific technical tasks such as resume parsing, job matching, and career recommendation.

a. Commercial tools

- **Job discovery**: Job discovery is mainly handled by platforms such as LinkedIn, Naukri, Indeed, Glassdoor, and company career pages. These platforms provide search filters, saved alerts, and job recommendations. However, candidates still have to read job descriptions manually and decide whether each role is suitable for them.
- **Career guidance:** Career guidance is often provided through static guides, skill taxonomies, or simple recommendation tools. These suggestions are usually

generic and based on job roles, not on the individual candidate's actual skills, projects, education, and experience.

- **Resume and application support:**
  Resume builders and general-purpose AI tools can help users write resumes, tailor resumes, or generate cover letters. However, in this project, resume tailoring and cover letter generation are treated as **optional good-to-have features**, not mandatory core features.

b. Academic Approaches

- **Resume Parsing:** Resume parsing is commonly treated as an information extraction task. Transformer models like BERT can be used to extract skills, education, experience, projects, and job titles from resume text. For resumes with complex layouts, models like LayoutLM can also consider document structure.
- **Resume Classification and Profile Understanding:** Recent work such as ResumeAtlas shows that large resume datasets and LLM-based methods can help classify resumes and understand candidate profiles. It also highlights challenges such as privacy, different resume formats, and data quality.
- **Job Matching:** Traditional job matching often depends on keyword overlap, but this may miss related skills or similar meanings. Semantic embedding models like Sentence-BERT can compare resumes and job descriptions based on meaning. Other person-job fit models, such as Qin et al. and ConFit, also explore neural and contrastive learning methods for better resume-job matching.
- **Career Recommendation:** Career recommendation can use recommender systems, knowledge graphs, and skill-role taxonomies. Research such as Dave et al. studies relationships between jobs and skills. Public taxonomies like O*NET and ESCO can help connect skills, occupations, and possible career paths.

4. Common Metrics and Existing Definitions of Success

Existing work uses different metrics depending on the stage of the workflow.

Metric Group 1: Quantitative Metrics

a. Precision, Recall, and F1-Score
Applied to **resume extraction**. These metrics check how accurately the system extracts structured details such as skills, education, experience, projects, and job titles from a resume. The extracted results can be compared with manually checked ground truth data.

b. Top-K Accuracy and Mean Reciprocal Rank (MRR)
Applied to **career path prediction**. Top-K Accuracy checks whether the correct or suitable career role appears in the top suggested roles. MRR checks how high the first correct recommendation appears in the ranked list.

Metric Group 2: Qualitative Metrics

a. LLM-as-Judge Scoring
Applied to **career recommendations**. A separate language model can rate the output on a 1 to 5 scale using criteria such as relevance, clarity, personalization, factual accuracy, and role fit.

b. Human Review
Human reviewers can check whether the tailored resume and cover letter are

specific, professional, and based only on the candidate's real profile. This is important because generated text should not include fake skills or unsupported claims.

How Success is Defined in Existing Systems

Academic systems measure success using proxy evaluation metrics such as F1-score for resume parsing and Top-k Accuracy/MR for career prediction, as real hiring outcomes are typically unavailable. Commercial platforms evaluate success using user and business outcomes, including application conversion, interview callback rates, and time-to-hire. For this project, success is defined by accurate resume parsing, complete candidate profile creation, relevant career recommendations, and useful job ranking. Optional features such as ATS scoring, resume tailoring, and cover letter generation will be evaluated only if implemented.

5. Where Current Solutions Fall Short and Opportunity for This Work

The main problem with current job search tools is that they are disconnected. Candidates use different platforms for job search, career guidance, resume support, and application preparation. Since these tools do not work together, the candidate has to manually connect the information and make decisions.
This creates the following gaps that our project can address:

- **Job discovery is limited to major job boards.**
  Most candidates search only on popular platforms like LinkedIn, Naukri, Indeed, and Glassdoor. These platforms show many jobs, but candidates still need help identifying which jobs are suitable for their profile. Our project will focus on finding and ranking relevant jobs based on the candidate's structured profile.

- **The workflow is fragmented.**
  Resume parsing, career recommendation, and job search are usually handled separately. The candidate has to copy information between tools and compare roles manually. Our project addresses this by creating one shared candidate profile that can be used for role recommendation and job ranking.

- **Career guidance is often generic.**
  Many career recommendation tools give general suggestions based on job titles or broad skill categories. They may not fully consider the candidate's actual education, projects, experience, and skills. Our system will provide role suggestions based on the candidate's extracted profile.

- **Limited profile-based feedback.**
  Candidates often do not know why a job is suitable or unsuitable for them. Our project will try to show basic reasons for job ranking, such as matching skills, experience level, and role relevance.

- **Application support is usually separate.**
  Resume tailoring, ATS scoring, and cover letter generation are useful features, but they are often handled by separate tools. In this project, these will be treated as optional good-to-have features if time permits.

Overall, the opportunity for our project is to build a connected, profile-based, system that supports the candidate from job discovery to tailored application material in one workflow.

6. Nature of the Project Contribution

The contribution of this project is not a claim of beating academic benchmarks. It is positioned as a practical system contribution across following dimensions:

- **Integration and usability:** The core system combines resume parsing, candidate profile creation, career role prediction, and job ranking in one connected workflow. This reduces the need for candidates to manually move between different tools.
- **Profile-based recommendations:** The system uses the structured candidate profile to suggest suitable career roles and rank jobs. This makes the output more specific to the candidate's skills, education, projects, and experience.
- **Explainable job support:** The system will not only rank jobs but also give basic reasons for the match, such as matching skills, relevant experience, or role similarity. This helps the candidate understand why a role is recommended.
- **Optional application support:** If time permits, ATS-style scoring, resume tailoring, and cover letter generation will be added as good-to-have features. These outputs will be based only on the candidate profile and job description to avoid fake or unsupported information.

Therefore, the main contribution lies in building a simple, usable, profile-based job search support system that helps candidates move from resume upload to career role recommendation and job ranking in one workflow.

7. Proposed System Architecture / Workflow

We have three decided modules for the project.

i. Resume Parsing
ii. Career Recommendation
iii. Job Discovery and job matching

A. Overall system overview

The user first uploads a resume. The **Resume Parsing module** extracts important details such as skills, education, experience, projects, certifications, and job titles. These details are stored in a structured candidate profile.

This structured profile is then used by the **Career Recommendation module** to suggest suitable career roles with short explanations.

The **Job Discovery and Job Matching module** uses the same profile and recommended career from career recommendation module to find relevant job opportunities and rank them based on similarity with the candidate's skills and background.

To keep the system secure and practical, authentication and privacy protection will also be included.

Model Choice: The project standardizes on Google's Gemini API (text and embedding models) throughout the pipeline. Gemini was selected over alternatives because it offers

a generous free-tier suitable for a student project budget, a single provider for both text generation and embeddings

Realistic example of process

A candidate uploads a resume containing Python, SQL, Excel, Power BI, and a dashboard project. The Resume Parsing module extracts these skills into JSON. The Career Recommendation module maps the profile to roles and recommends Data Analyst, BI Analyst, and Reporting Analyst. The Job Matching module compares the profile with available job descriptions and ranks a Junior Data Analyst role highest because it matches SQL, dashboarding, and entry-level experience.

Authentication

The system will use Google SSO for login, so the application does not need to store or handle user passwords. The backend will verify Google's signed id_token on the server side by checking the signature, audience, issuer, and expiry.

After successful login, the session will be managed using HttpOnly, Secure, and SameSite cookies. This helps reduce common risks such as XSS and CSRF. Since Google login is used, the system can also benefit from Google's existing MFA and account recovery features.

Database

- PostgreSQL for user profile, parsed resume fields, job postings, evaluation results.
- pgVector/ChromaDB for candidate and job embeddings.

Computational Requirements

The system is designed to run on lightweight cloud infrastructure rather than local GPU hardware. The web application and PostgreSQL/vector database will be hosted on a small cloud instance like EC2 instance sized for a prototype, not production load). All LLM and embedding operations use Gemini's hosted API on its free or low-cost tier, so no local model hosting or GPU is required. This keeps the project's infrastructure cost near zero

Prompt Engineering, Caching, and Latency

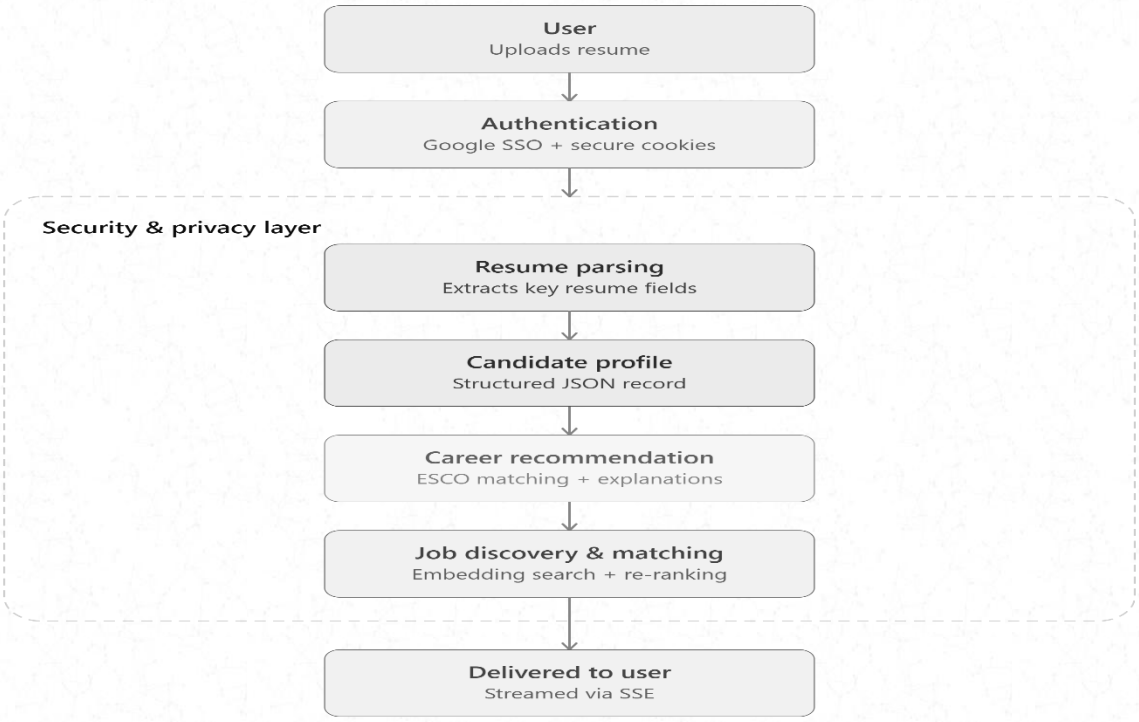
All LLM calls use structured prompting with a fixed output JSON schema to keep outputs parseable and reduce hallucination.

To control cost and latency, embeddings for a given resume or job posting are computed once and cached (keyed by a content hash) in the vector store, so repeat matches against the same resume/job do not trigger redundant API calls.

Security and Privacy Measures

- **PII redaction before LLM usage:**
  Sensitive details such as name, email, and phone number will be masked before sending resume content to the LLM. Tools such as Microsoft Presidio can be used for this. The original details can be reinserted locally after processing.

- **Safe file upload handling:**
  Uploaded PDF or DOCX files will be validated and processed in a controlled environment to reduce risks from malicious files or path traversal attacks.
- **Safe job discovery:**
  Job discovery will use only public pages where allowed. Company career pages or portals protected by anti-bot mechanisms, login walls, CAPTCHA, or restrictive Terms of Service will be excluded from automated collection rather than bypassed.
- **Prompt injection control:**
  Resume text and job descriptions will be treated as untrusted input. They will be passed as data-only context, and the model output will be restricted to a fixed JSON schema where possible.



B. Module-wise analysis:

i. Parsing Modules

a. Purpose:

The purpose of this module is to convert an uploaded resume into a structured candidate profile. This profile will act as the main input for the other modules.

b. Research Basis:

Resume parsing is commonly treated as an information extraction task. Transformer-based models such as BERT are useful because they learn contextual meaning from text and can be fine-tuned for tasks such as entity

extraction. For resumes with complex layouts, models such as LayoutLM are useful because they consider both text and layout information, which helps in document understanding.

c. **Proposed Approach:**
The uploaded resume will first be converted into text. Then an LLM will extract the required fields and return them in a structured JSON format.

d. **Input:**
Uploaded resume in PDF, DOCX, or text format.

e. **Processing Steps:**
1. Extract raw text from the uploaded resume.
2. Clean and preprocess the text.
3. Use an LLM prompt to extract fields such as:
    - skills
    - education
    - work experience
    - projects
    - certifications
    - job titles
4. Store the extracted details as a structured candidate profile.

f. **Output:**
A structured JSON candidate profile.

g. **Evaluation:**
This module will be evaluated on a manually curated set of 20–25 resumes drawn from the Kaggle Resume Dataset. The ground-truth fields such as skills, education, experience, projects, certifications, job titles will be manually annotated for each resume. Extracted output is then compared against this reference using precision, recall, and F1-score per field type.

ii. **Career Recommendation Module**

a. **Purpose:** The purpose of this module is to recommend suitable career roles and job titles based on the structured candidate profile.

b. **Research Basis:** This module is grounded in Semantic Similarity Search over the ESCO skill-role taxonomy. The ESCO dataset provides the reference taxonomy of occupations, skills, knowledge, and abilities against which candidate profiles are matched.

This module grounds every recommendation in ESCO's structured taxonomy, enabling traceable and explainable skill-gap analysis rather than free-form LLM inference.

c. **Proposed Approach:** This module uses the structured candidate profile generated by the Resume Parsing module as input. The approach follows five main stages, and an evaluation stage:

Phase 1: Input Processing

- **Input:** Structured JSON candidate profile.
- **Tool:** LangChain (Data Parser).
- **Action:** The system reads the candidate profile and extracts important fields such as skills, education, experience, and projects. These fields are combined into a single candidate context.

Phase 2: Candidate Vectorization

- **Tool:** Gemini API (Embedding Model)
- **Action:** The candidate context is converted into a dense vector representation that mathematically encodes the candidate's professional profile.

Phase 3: Semantic Similarity Search

- **Tool:** ChromaDB
- **Action:** The candidate vector is compared with pre-stored ESCO occupation vectors using cosine similarity. ChromaDB returns the Top-K closest career roles. Chroma is suitable here because it supports vector search over embeddings.

Phase 4: LLM-Based Explanation

- **Tool:** LLM text model with prompt chaining.
- **Action:** The LLM generates a structured JSON output containing an explanation for each recommended role and a list of missing_critical_skills, selected only from the ESCO skill taxonomy associated with that occupation to minimize hallucinated or unsupported claims.

Phase 5: Output Delivery

- **Tool:** Server-Sent Events (SSE).
- **Action:** To avoid blocking the frontend during the multi-stage AI pipeline, the system uses *Server-Sent Events (SSE)*. The backend computes results and pushes them to the client. The backend streams progress updates such as "*Analysing profile*," "*Matching with ESCO roles*," and "*Generating explanations*," followed by the final structured JSON output once the LLM Reasoner completes.

Phase 6: Evaluation and Success Criteria

- **Ground Truth Construction (One-time):** A test set of approximately 25–30 candidate profiles are manually curated. For

each profile, acceptable job matches are labelled by mapping them to specific ESCO occupation IDs. This ground truth is built once and reused across all subsequent evaluation runs.

- **Retrieval Quality:** Retrieval is scored by comparing returned *ESCO occupation IDs* directly against the ground-truth ID set. The % of candidates for whom at least one ground-truth ESCO ID appears in the top-K ChromaDB results should be greater than **75%** for this module to be treated as a success.

iii. Job Discovery and job matching

a. Purpose: The purpose of this module is to discover relevant job opportunities and rank them according to the candidate's structured profile. It uses the candidate's skills, experience, preferred role, location, and job type to find jobs that are more suitable for the candidate.

b. Proposed Approach

Phase I: Input Processing

Input:

Structured JSON candidate profile from parser module and recommended career from career recommendation module.

Action:

The system reads the candidate profile and creates a search context.

Output:

A clean candidate search query and candidate profile object for job matching.

Phase II: Job Pool Creation

Input:

Candidate target role, skills, experience, and location.

Tool:

Internal job database, public job datasets, saved job postings, and allowed public company career pages.

Action:

The system collects job postings and converts them into structured form.

Each job posting may include:

- job title
- company
- location
- required skills
- experience required
- employment type
- job description
- posting date

Output:
A structured job postings pool.

This pool shall be stored in database layer it will make system reusable and remove unnecessary calling of the API.

Phase III: Semantic Job Matching

Tool:
Embedding model such as Gemini Embeddings and vector database such as pgvector, FAISS, or ChromaDB.

Action:
The candidate profile and job descriptions are converted into embeddings. The system compares them using cosine similarity and retrieves the Top-K matching jobs.

Why:
Embeddings help match jobs by meaning, not only by exact keywords. For example, "data visualization" and "dashboard building" may be treated as related even if the exact words are different.

Output:
Top matching jobs with similarity scores.

Phase IV: Rule-Based Re-Ranking

Tool:
Rule-based ranking engine.

Action:
The semantically matched jobs are re-ranked using practical ranking signals.

Ranking Signal	Description	Weight
Skill Match	How well the job skills match the candidate profile	0.50
Experience Fit	How closely the candidate's experience matches the job requirement	0.25
Employment Type Match	Whether job type matches preference such as full-time, internship, remote, etc.	0.20
Job Freshness	Gives small preference to recently posted jobs	0.05

Output:
A final ranked list of jobs.

Phase V: Web Search Fallback

Trigger:
This phase is used only when the internal job pool** does not return enough good matches.

Tool:
SearXNG / public web search fallback.

Action:
The system creates a search query using the candidate's role, skills, experience, and location. It then searches public job pages and filters duplicate or irrelevant results.

***job database pool created and stored in phase II*

Important Limitation:
The system will use only public pages where allowed. It will not bypass login pages, CAPTCHA, or restricted job boards.

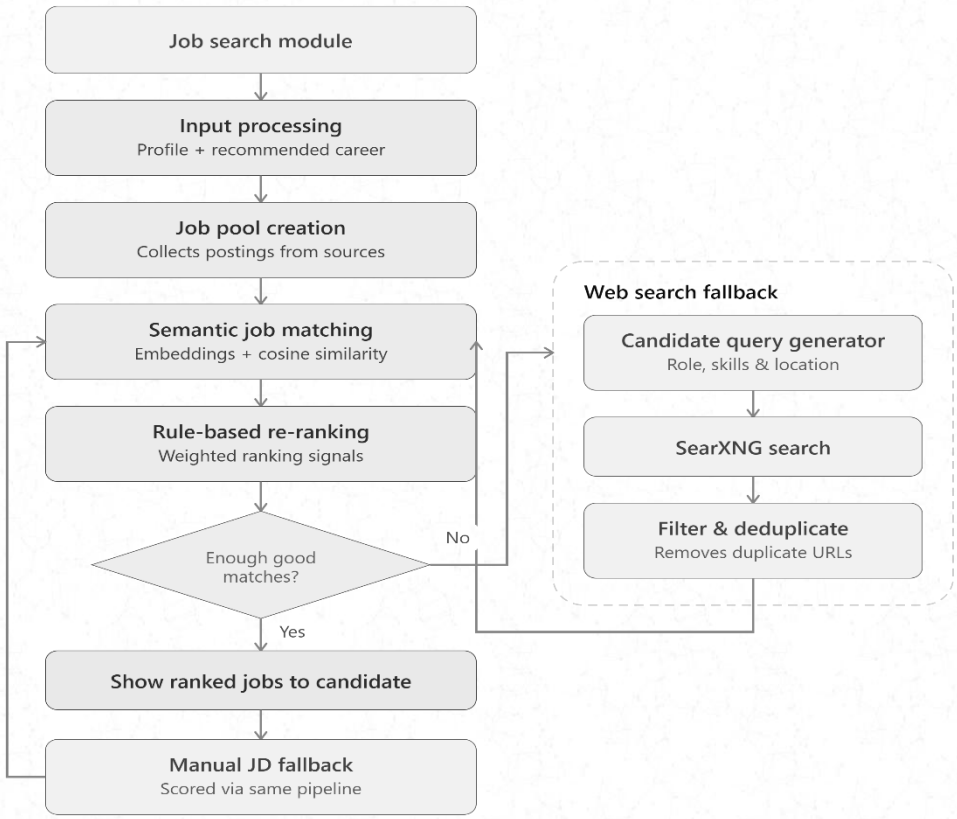
Output:
Extra public job results, which are again passed through the same matching and ranking process.

Phase VI: Manual Job Description Fallback

Input:
Candidate pasted job description.

Action:
If automated job discovery is limited, the candidate can paste a job description manually. The pasted job description will be scored using the same matching pipeline.

Why:
This makes the module more reliable because the system can still work even if live job discovery is unavailable.



c. Evaluation

This module can be evaluated using a small set of candidate profiles and job descriptions. Following data is available for that.

- Candidate profiles: Kaggle — Candidate Job Role Dataset
- Job postings: Kaggle — LinkedIn Job Postings Dataset

Each job can be manually labelled as:

- good match
- partial match
- poor match

The module can be checked using simple ranking metrics such as:

- **Recall@100:** Checks whether relevant jobs are retrieved.
- **NDCG@10:** Checks whether better jobs appear near the top.
- **Precision@20:** Checks how many of the top jobs are actually relevant.

d. Reason for selection of SearXNG:

SearXNG is a self-hosted, open-source metasearch engine. Instead of calling a single search provider directly, it aggregates results from many underlying engines behind one internal interface.

Problem with Direct Search APIs	How SearXNG Mitigates It
Rate limits per provider (e.g., Google Custom Search, Bing Search API cap queries/day or require paid tiers)	Queries are distributed across multiple backend engines, so no single provider's quota becomes a hard bottleneck
API availability / outages	If one backend engine is down or throttled, SearXNG can be configured to fail over to others without the orchestrator noticing
Cost at scale	Most commercial search APIs charge per query; self-hosted SearXNG removes per-query licensing cost, at the expense of running/maintaining infrastructure
Query privacy	Self-hosting avoids sending raw candidate-derived queries to third-party search vendors, which matters given PII in the pipeline

8. Preliminary Dataset Plan and Challenges

The project does not depend on a single dataset. Instead, different data sources will be used for different parts of the pipeline.

Particulars	Expected Data Source	Purpose
Resume parsing	Kaggle Resume Dataset / open-source resume samples / anonymized student resumes	To extract skills, education, experience, projects, certifications, and job titles
Candidate profile creation	Output from Resume Parsing module	To create a structured candidate profile used by all modules.
Career Recommendation	ESCO Occupations + ESCO Skills taxonomy	To map candidate skills and experience to suitable job roles
Job discovery and matching	Public job description datasets, saved job postings, company career pages, job aggregators were allowed	To collect job title, company, location, required skills, experience, job type, and description
Evaluation data	Small, curated set of resumes, job descriptions, expected role suggestions, tailored resumes, and cover letters	To evaluate output quality and system usefulness

Till now we have identified some of databases, we will use in training and testing of our projects.

ESCo Skills-Occupations CSV Dataset

https://www.kaggle.com/datasets/ckshetty/candidate-job-role-dataset

https://www.kaggle.com/datasets/arshkon/linkedin-job-postings

Kaggle Resume Dataset

Anticipated Challenges

Data availability:
There may not be one complete dataset that contains resumes, job descriptions, career roles, ATS scores, and cover letters together. Therefore, the project will combine multiple smaller datasets and manually curated examples.

Privacy:
Real resumes may contain personal information. Any volunteer resumes used in the project should be anonymized before processing.

Dynamic job postings:
Online job posts change frequently. Some postings may expire or be removed. For the prototype, saved job descriptions or public datasets can be used to keep evaluation stable.

9. Evidence That Will Demonstrate the Approach Works

Component	Evidence of Success	Metric / Target
Resume Parsing	The system should extract skills, education, experience, projects, certifications, and job titles from uploaded resumes in structured JSON format.	Precision, Recall, F1-Score for extracted fields. Target: F1-score ≥ 75% on a small manually checked resume set.
Candidate Profile Creation	The extracted resume details should be converted into one clean candidate profile that can be reused by other modules.	Profile completeness score based on required fields. Target: 80% or more required fields filled correctly for valid resumes.
Career Recommendation	The system should recommend suitable career roles based on the candidate's skills, education, experience, and projects, with short explanations.	Top-5 Accuracy / Hit Rate@5. Target: at least one suitable role appears in the top 5 recommendations for 80% of test profiles.
Job Discovery	The system should collect or use job descriptions with important fields such as title, company, location, required skills, experience, and job type.	Job field extraction accuracy. Target: 80% of collected jobs should have key fields extracted correctly.
Job Matching and Ranking	The system should rank jobs according to candidate-profile fit,	NDCG@10 / Precision@10. Target: NDCG@10 ≥ 0.60 or Precision@10 ≥ 40% on

Component	Evidence of Success	Metric / Target
	so better-matching jobs appear higher in the list.	manually labelled resume-job pairs.
Web Interface	The user should be able to upload a resume, view the structured profile, see career recommendations, and view ranked job matches.	Task completion test. Target: user completes the full flow without error in at least 80% of test runs.

10. Known limitations and constraints

- An application record is a proxy for fit: it means the user was interested, not that the job matched or that they were hired.
- Career recommendations are grounded in ESCO, which is EU-centric and slow to update, so fast-moving or region-specific roles may map poorly or be missing.
- Resume parsing accuracy may vary for highly designed or image-based resumes.
- Live job postings may expire or change.
- Web scraping is limited by website structure changes, and legal restrictions.
- Career recommendations may be biased if candidate data or taxonomy coverage is limited.
- Job ranking is a decision-support score, not a guarantee of hiring success.
- Keyword and skill overlap can't judge proficiency — matching "Python" to "Python" says nothing about depth, so a keyword-heavy resume can score close to a stronger one.

11. Team consent

Name	Roll Number	Initial
Gaurav Kumar	22f1001105	GK
Dev Gupta	22f2000888	DG
Abhinav Ohri	24f1002064	AO
Pranav N	22f2000117	PN
Praveena N	22f3001454	PN